



INDIA.RUNS

Build what next India runs on

Team Name :Ayesathon

Team Leader Name :Ayesha Siddiq

Problem Statement : Rank candidates by relevance using profile and platform signals and produce a validated top-100 CSV that meets the contest validator rules.

Solution Overview

- **Heuristic + Model Ensemble:** a handcrafted weighted heuristic captures clear profile signals, a LightGBM regressor is trained on those heuristic scores (pseudo-targets), and the final ranking is an average of heuristic + model outputs.
What differentiates your approach from traditional candidate matching systems?
- **Combines transparent, interpretable signals with learned nonlinear interactions; enforces strict validator rules and deterministic tie-breaks for reproducibility.**

JD Understanding & Candidate Evaluation

- **Signals used:** profile completeness, years of experience, skill match score, recruiter response rate, GitHub/activity, profile views, connections.
- **Beyond keywords:** compute normalized skill vectors, weigh recent experience higher, combine recruiter response and activity signals, and apply sanity checks for inconsistent data.

Ranking Methodology

- Heuristic weighted score (normalized features) + LightGBM regressor trained on heuristic pseudo-targets (fast CV).
- How are multiple signals combined into a final ranking?
- $\text{FinalScore} = \text{average}(\text{HeuristicScore}, \text{ModelPrediction})$; scores rounded to 4 decimals, sorted descending, tie-broken by `candidate_id`.

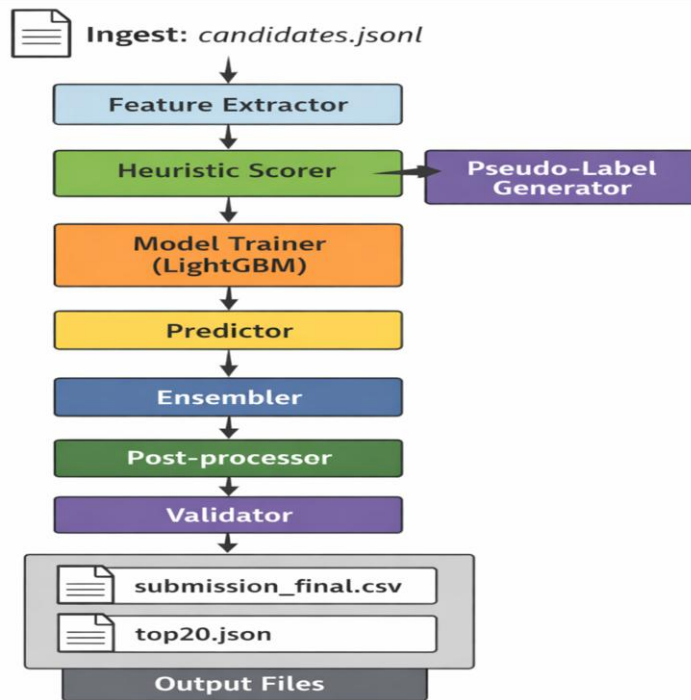
Explainability & Data Validation

- Show top 3 contributing signals per candidate (e.g., completeness + skill_match + recruiter_response) and a one-line justification derived from profile fields.
How do you prevent hallucinations or unsupported justifications?
- Only surface facts present in profile fields; justifications reference explicit fields (skill names, experience years, response rate).
How do you handle inconsistent/low-quality/suspicious profiles?
- Downweight incomplete profiles, flag suspicious records for manual review, and exclude entries failing basic schema checks.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?
 1. JD → extract required skill keywords and constraints.
 2. Load candidates.jsonl → feature extraction.
 3. Compute heuristic scores → generate pseudo-labels.
 4. Train LightGBM on pseudo-labels (optional fast CV).
 5. Ensemble heuristic + model, post-process (round, sort, tie-break).
 6. Run validator → produce [submission_final.csv](#) and package zip.

System Architecture



Single-Machine Python Pipeline

Optional Web Demo (demo.html)

Results & Performance

- Final deliverable: [submission_final.csv](#) (validated).
- Proxy improvements: ensemble improves top-100 average recruiter_response and completeness vs baseline (demonstrated in repo scripts).

How does your solution meet runtime/compute constraints?

Lightweight pipeline (pandas + LightGBM) runs on a standard laptop in under 10 minutes for dataset processing and training.

Technologies Used

- Python 3.13, pandas, numpy, scikit-learn, LightGBM
- python-pptx (slides), PyMuPDF (template extraction), simple HTML demo (demo.html)
- Packaging: zip for submission artifacts

Submission Assets

- GitHub repo: <https://github.com/Ayeshasiddiqa789/india-runs-hackathon>
- Files to attach:
[submission_final.csv](#), submission_final_package.zip (or submission_package.zip),
India_Runs_Presentation_from_template.pptx
- Demo: [top20.json](#) + [demo.html](#) (or hosted demo/video link)
- Repro steps: See README in [hackathon solution](#) for commands to regenerate artifacts



INDIA.RUNS

Build what next India runs on

THANK YOU